

Esame OS Internals (Cabodi)

Data: 12/09/2020

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale e LRU)

Contesto: Si consideri, per un dato processo, la seguente stringa di accessi alla memoria. Per ogni indirizzo (indirizzamento Byte, con indirizzi espressi in codice esadecimale) viene riportata anche l'operazione di lettura (R)/scrittura (W): R 33F5, R 2A64, W 0AD3, W 2E7E, R 08C8, W 13D1, R 094E, R 3465, W 32A0, R 1BBA, W 0CE6, R 1480, R 3294, R 2AB8. Supponiamo che gli indirizzi fisici e logici siano rappresentati su 16 bit, la dimensione della pagina sia 2 KByte e 3C02 sia l'indirizzo massimo utilizzabile dal programma (il limite superiore dello spazio degli indirizzi).

Si risponda alle seguenti domande:

-) Calcolare la dimensione dello spazio di indirizzamento (espressa come numero di pagine) e la frammentazione interna.
 -) Calcolare la stringa degli accessi alle pagine.
 -) Simulare un algoritmo di sostituzione delle pagine di tipo LRU (Least Recently Used), con 3 frame disponibili. Rappresentare il resident set dopo ogni accesso alla memoria. Indicare i page fault.
-

SECTION 2: OS161

Domanda 2 (Processi e Trapframe)

Si consideri un processo utente su OS161:

-) Spiegare brevemente la differenza tra `switchframe` e `trapframe`. Quale viene utilizzato per avviare un processo utente? Quale per gestire una chiamata di sistema? Quale per la `thread_fork`?
 -) Lo `switchframe` è allocato nello stack utente o nello stack di processo a livello di kernel? (Motivare).
 -) Perché i dispositivi di I/O sono mappati su `kseg1`? Sarebbe possibile mappare un dispositivo di I/O in `kseg0`?
 -) Un processo utente può eseguire una `write(fd, buf, nb)`, dove `buf` è un indirizzo in `kseg0`?
-

Domanda 3 (Gestione Console e Condition Variables)

Contesto: Si consideri la gestione della console in OS161.

Si risponda alle seguenti domande:

-) Quale delle due funzioni (`putch_intr` o `con_start`) viene chiamata per gestire l'output sulla console seriale? (Motivare).
 -) Fornire la definizione del campo `cs_wsem` nella `struct con_softc`.
 -) Fornire un'implementazione delle funzioni di gestione console sostituendo il semaforo `cs_wsem` con una condition variable.
-

Domanda 4 (Strutture Thread e Processi)

Contesto: Si consideri la relazione tra `struct thread` e `struct proc` in OS161.

Si risponda alle seguenti domande:

-) Si dica come sono correlate/collegate le due struct nel caso in cui un processo abbia più di un thread.
-) Si dica anche se, dato un puntatore `p` a una `struct proc`, sia possibile ricavare l'elenco di tutti i thread del processo e stamparne il nome. Qualora non sia possibile, si proponga come modificare la `struct proc` e/o `struct thread`, in modo da poter realizzare l'operazione. Scrivere la funzione `printThreadNames(struct proc *p)` che svolge tale compito.
-) Si spieghi brevemente il ruolo del puntatore `t_context` nella `struct thread`.

ANSWERS

Domanda 1 (Memoria Virtuale e LRU)

A) Dimensione dello spazio di indirizzamento e frammentazione interna:

L'indirizzo massimo utilizzabile è 0x3C02, che in decimale corrisponde a 15362. Poiché gli indirizzi di memoria partono dal byte 0, lo spazio totale occupato dal programma è di $15362 + 1 = 15363$ Byte.

- **Numero di pagine:** La dimensione della pagina è $2 \text{ KB} = 2048 \text{ Byte}$.
Lo spazio di indirizzamento in pagine è $\lceil 15363 / 2048 \rceil = 8$ pagine.
- **Frammentazione interna:** È data dalla differenza tra la memoria fisica allocata (8 pagine intere) e la memoria effettivamente richiesta dal programma.
 $(8 \cdot 2048) - 15363 = 16384 - 15363 = 1021 \text{ Byte}$.

B) Stringa degli accessi alle pagine:

Avendo pagine da 2 KB (2^{11} Byte), i primi 11 bit meno significativi (LSB) dell'indirizzo logico costituiscono l'offset, mentre i restanti bit più significativi (MSB) indicano il numero di pagina virtuale. Per ricavare la pagina, è sufficiente dividere l'indirizzo decimale per 2048 e prenderne la parte intera (o traslare a destra di 11 bit):

- R 33F5 (13301) → Pagina 6
- R 2A64 (10852) → Pagina 5
- W 0AD3 (2771) → Pagina 1
- W 2E7E (11902) → Pagina 5
- R 08C8 (2248) → Pagina 1
- W 13D1 (5073) → Pagina 2
- R 094E (2382) → Pagina 1
- R 3465 (13413) → Pagina 6
- W 32A0 (12960) → Pagina 6
- R 1BBA (7098) → Pagina 3
- W 0CE6 (3302) → Pagina 1
- R 1480 (5248) → Pagina 2
- R 3294 (12948) → Pagina 6
- R 2AB8 (10936) → Pagina 5

La stringa degli accessi è: **6, 5, 1, 5, 1, 2, 1, 6, 6, 3, 1, 2, 6, 5**.

C) Simulazione algoritmo LRU (3 frame disponibili):

La politica *Least Recently Used* (LRU) seleziona per lo sfratto la pagina che non è stata acceduta da più tempo.

R/W	Indirizzo	Pagina	Hit/Miss	Sfratto	Resident Set
R	33F5	6	Miss (1)	-	[6]
R	2A64	5	Miss (2)	-	[6, 5]
W	0AD3	1	Miss (3)	-	[6, 5, 1]
W	2E7E	5	Hit	-	[6, 5, 1]
R	08C8	1	Hit	-	[6, 5, 1]
W	13D1	2	Miss (4)	6	[2, 5, 1]
R	094E	1	Hit	-	[2, 5, 1]
R	3465	6	Miss (5)	5	[2, 6, 1]
W	32A0	6	Hit	-	[2, 6, 1]
R	1BBA	3	Miss (6)	2	[3, 6, 1]
W	0CE6	1	Hit	-	[3, 6, 1]
R	1480	2	Miss (7)	6	[3, 2, 1]
R	3294	6	Miss (8)	3	[6, 2, 1]
R	2AB8	5	Miss (9)	1	[6, 2, 5]

Totale Page Faults: 9

Domanda 2 (Processi e Trapframe)

A) Switchframe vs Trapframe e il loro utilizzo:

- **Trapframe:** Salva l'intero contesto della CPU (tutti i registri). Viene generato dall'hardware in seguito a un'eccezione, un interrupt o una system call, gestendo la transizione asincrona tra *User Mode* e *Kernel Mode*.
- **Switchframe:** Salva **esclusivamente** i registri *callee-save* (richiesti dalla convenzione di chiamata C). Viene generato via software durante un *Context Switch* volontario tra due thread già in esecuzione all'interno del kernel.

Utilizzo nei casi specifici:

- **Avvio processo utente: Trapframe** (si forza un ritorno da eccezione fittizio verso lo user space).
- **Gestione system call: Trapframe** (innescata da eccezione software).
- **thread_fork: Switchframe** (prepara il contesto minimo per il primo context switch del nuovo thread).

B) Locazione dello Switchframe:

Lo switchframe è allocato nello **stack del kernel** (kernel stack) del thread.

Motivazione: Il context switch è un'operazione privilegiata che avviene interamente in Kernel Mode. Salvare i registri sensibili della CPU nello stack utente rappresenterebbe una grave falla di sicurezza, esponendo il sistema a manipolazioni da parte dei processi utente.

C) Dispositivi di I/O e kseg1:

I dispositivi hardware (Memory-Mapped I/O) sono mappati su **kseg1** perché è un segmento di

memoria **non soggetto a caching** (uncached).

Non sarebbe possibile mapparli in `kseg0` (che è *cached*): le letture preleverebbero dati vecchi dalla cache invece dello stato reale del dispositivo, e le scritture (comandi) rimarrebbero bloccate nella cache della CPU senza mai raggiungere l'hardware periferico.

D) Buffer `kseg0` in system call write:

NO. Il segmento `kseg0` è riservato al kernel. Il Sistema Operativo controlla rigorosamente i puntatori passati dall'utente (es. tramite `copyin/copyout`) e, rilevando un indirizzo fuori dal `kuseg` (User Space), farà fallire la system call restituendo un errore (es. `EFAULT`) per proteggere la memoria privilegiata.

Domanda 3 (Gestione Console e Condition Variables)

A) Funzioni per l'output seriale:

- **con_start:** Viene chiamata dal software del kernel per **iniziare** il trasferimento di un carattere, scrivendolo nell'apposito registro hardware del dispositivo seriale.
- **putch_intr:** Viene chiamata **in seguito** dall'hardware (come gestore dell'interrupt) per notificare alla CPU che il dispositivo ha terminato la trasmissione del carattere ed è pronto a riceverne uno nuovo.

B) Campo `cs_wsem`:

Nella `struct con_softc`, il campo `cs_wsem` è un **semaforo** utilizzato per la sincronizzazione dell'output. Ha un doppio ruolo: garantisce la mutua esclusione tra thread concorrenti che vogliono scrivere sulla console, e blocca (*sleep*) il thread corrente finché l'interrupt handler (`putch_intr`) non segnala (*wake*) che il dispositivo è libero.

C) Implementazione con Condition Variable:

Per sostituire il semaforo, introduciamo una CV, un Lock e un flag di stato nella struttura.

```
/* Struttura */
struct con_softc {
    struct cv *cs_wcv;
    struct lock *cs_wlock;
    bool cs_writing;
};

/* Funzione di scrittura (es. con_start logic) */
void console_write_char() {
    lock_acquire(sc->cs_wlock);
    while (sc->cs_writing) {
        cv_wait(sc->cs_wcv, sc->cs_wlock);
    }
    sc->cs_writing = true;
    /* Invia carattere al registro hardware */
    lock_release(sc->cs_wlock);
}

/* Gestore interrupt (putch_intr logic) */
void putch_intr() {
    lock_acquire(sc->cs_wlock);
```

```

    sc->cs_writing = false;
    cv_signal(sc->cs_wcv, sc->cs_wlock);
    lock_release(sc->cs_wlock);
}

```

Domanda 4 (Strutture Thread e Processi)

A) Correlazione tra struct thread e struct proc:

In OS161, la correlazione nativa è gestita **esclusivamente dal thread**. Ogni **struct thread** possiede un puntatore diretto (**t_proc**) alla **struct proc** del processo a cui appartiene. Al contrario, nell'implementazione di base, la **struct proc** non possiede alcun riferimento inverso (né liste né array) per tenere traccia dei thread che le appartengono.

B) Ricavare l'elenco dei thread e modifica strutturale:

NO, non è possibile con l'implementazione nativa, proprio per l'assenza di riferimenti all'interno di **struct proc**.

Modifica proposta: Aggiungere all'interno di **struct proc** un array di puntatori ai thread e un contatore:

```

struct proc {
    /* ... altri campi ... */
    struct thread *p_threads[MAX_THREADS];
    int p_numthreads;
};

```

Funzione di stampa:

```

void printThreadNames(struct proc *p) {
    if (p == NULL) return;
    for (int i = 0; i < p->p_numthreads; i++) {
        if (p->p_threads[i] != NULL) {
            kprintf("Thread %d: %s\n", i, p->p_threads[i]->t_name);
        }
    }
}

```

C) Ruolo del puntatore t_context:

Il campo **t_context** all'interno della **struct thread** serve a memorizzare il puntatore all'area di memoria (tipicamente sullo stack) in cui è salvato lo **switchframe** del thread nel momento in cui questo viene sospeso. È il meccanismo fondamentale del *Context Switch*: quando il dispatcher decide di rimettere in esecuzione il thread, utilizzerà **t_context** per ricaricare i registri salvati e riprendere l'esecuzione esattamente da dove si era interrotta.